

VIETNAM GENERAL CONFEDERATION OF LABOUR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



523c0012 – Huynh Nhat Huy
523v0002 – Bui Quang Huy

FINAL PROJECT — PART 2

SAIGONRIDE: DISTRIBUTED VEHICLE RENTAL SYSTEM

System Design Document

Introduction to Software Engineering | HK2, 2025-2026
Instructor: Ky-Trung Pham

Tier 4 — ASP.NET MVC Code-First + Entity Framework + VNPay Sandbox
(Max Score: 10.0)

HO CHI MINH CITY, YEAR 2025 - 2026

1. Overall UML Use Case Diagram

1.1. Overview

The Overall Use Case Diagram illustrates the functional scope of the SaigonRide - Distributed Vehicle Rental System. It serves as a visual map of the system's boundaries, defining how various external actors interact with the internal functionalities to achieve specific business goals. The diagram is organized into logical packages—Security, Administration, and Rental Management—to maintain architectural separation of concerns.

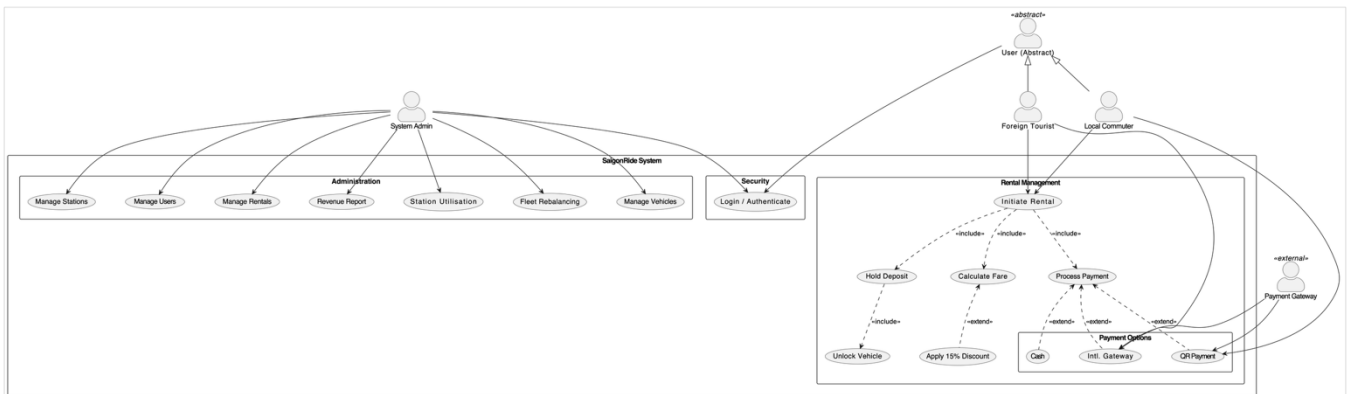


Figure 1.1: Overall Use Case Diagram for the SaigonRide System

1.2. Actor Definitions and Generalization

A critical component of this design is **Actor Generalization**, allows the system to manage a unified user base while catering to the unique requirements of different demographics.

- **User (Abstract):** Base actor representing common functionalities shared by users, primarily the authentication process.
- **Local Commuter:** A specific actor representing local residents who utilize local payment methods. Inherits all basic behaviors from the Abstract User.
- **Foreign Tourist:** A specific actor representing international visitors who require international payment gateways. Iso inherits from the Abstract User.
- **System Admin:** A specialized actor responsible for operations, such as the vehicle fleet, station inventory management, and system-wide reporting.
- **Payment Gateway:** An external system actor (e.g., VNPay or PayPal API) that interfaces with the platform to verify and process financial transactions.

1.3. Functional Packages and Relationships

The system's features are categorized to align to **MVC/3-Tier architecture** requirements:

Security: Centralizes Login / Authenticate use case, ensuring secure access for all actors.

Administration: Full **CRUD (Create, Read, Update, Delete)** operations for core entities, including Vehicles, Stations, Users, and Rentals. Also includes the

generation of two mandatory operational reports: Revenue Report and Station Utilization.

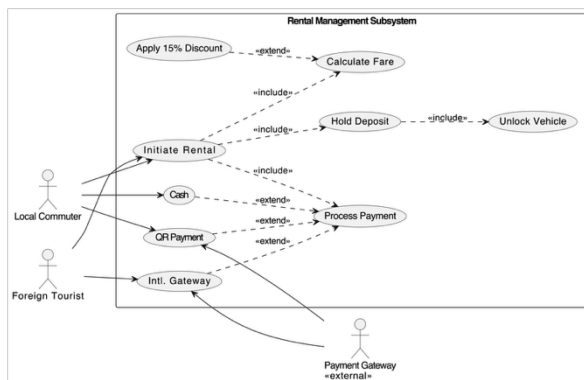
Rental Management: Handles core business logic of distributed rental network.

Include Relationships: Used for mandatory steps such as Hold Deposit and Calculate Fare which must occur during the Initiate Rental process.

Extend Relationships: Used for conditional logic, such as the **discount** applied only when a vehicle is returned to a low-inventory station. Also handles the choice of payment methods (QR, International, or Cash) based on the actor's preference.

1.4. Use Case Specifications & Behavioral Diagrams

1.4.1. UC1: Process Rental & Payment - Owner: Huynh Nhat Huy (523c0012)



Field	Detail
Use Case Name	Process Rental & Payment
Actor	Local Commuter, Foreign Tourist
Pre-conditions	User is successfully authenticated. At least one vehicle is "Available" at the selected station.
Post-conditions	A rental record is created, the vehicle status is updated, & finalized via the selected Payment Gateway.

Figure 1.2: Rental Management Subsystem Use Case Diagram. (Details core business logic including vehicle rental initiation, automated fare calculation, and multi-channel payment processing).

Main Success Scenario (Main Flow):

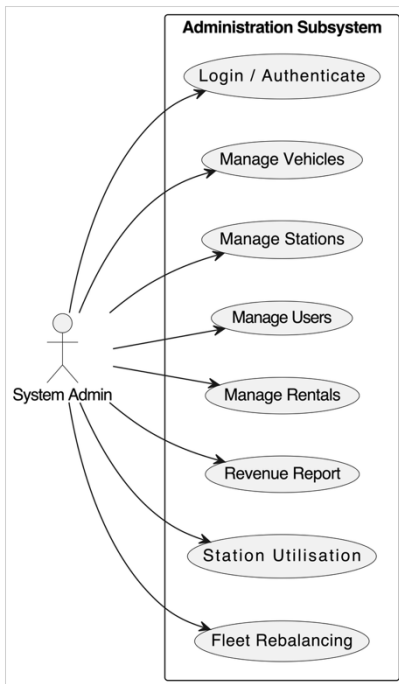
1. User selects an available vehicle at a station through the application.
2. System initiates a rental session and updates the vehicle status to "In-Transit".
3. System prompts the user to authorize a security deposit (e.g., via VNPay Sandbox).
4. User completes the deposit, and the system unlocks the vehicle.
5. User completes the trip and returns the vehicle to any target station.
6. System records return timestamp & calculates base fare based on vehicle category.
7. System checks the destination station's capacity.
8. System calculates final fare, apply discount if the station capacity is below 20%.
9. System processes the final payment and releases the remaining deposit.
10. System generates a rental summary and updates the station's inventory.

Extensions (Alternate Flows):

6a. Location-based Discount: If the return station is at <20% capacity, the system automatically applies a 15% discount to incentivize fleet rebalancing.

9a. Cash Payment: User selects "Cash" at a kiosk; the system prompts for manual deposit/payment handling.

1.4.2. UC2: Manage Vehicle Fleet - Owner: Bui Quang Huy (523v0002)



Main Success Scenario (Main Flow):

1. Admin logs into the Management Dashboard.
2. Admin selects "Vehicle Management" module.
3. System retrieves and displays a comprehensive list of all vehicles, including their current status (Available, In-Transit, Maintenance).
4. **Create:** Admin adds new vehicles by entering ID, category (Standard/E-Scooter), and initial station.
5. **Update:** Admin modifies an existing vehicle's status (e.g., moving a vehicle to "Maintenance") or reassigns it to a different station.
6. **Delete:** Admin removes a retired vehicle record from the active fleet.
7. System validates the changes and persists them to the SQL Server database using Entity Framework.
8. System confirms the operation success and refreshes the fleet view.

Figure 1.3: Administration Subsystem Use Case Diagram. (Outlines administrative privileges for managing system entities (CRUD operations) and generating operational reports.)

Field	Detail
Use Case Name	Manage Vehicle Fleet
Actor	System Admin
Pre-conditions	Admin is authenticated with administrative privileges.
Post-conditions	The vehicle database is updated (created, modified, or deleted) to reflect the physical fleet status.

Extensions (Alternate Flows):

- **4a. Validation Error:** If a duplicate Vehicle ID is entered, the system rejects the entry and displays a validation message.
- **5a. Mass Status Update:** Admin can filter vehicles by category to perform bulk status updates for maintenance cycles.

2. Sequence Diagrams

2.1. User Sequence Diagrams

· **Figure 2.1.1: Sequence Diagram for Rental Initiation and Vehicle Return**
(Traces the sequential interactions between the User, Kiosk interface, and Backend System during the rental lifecycle.)

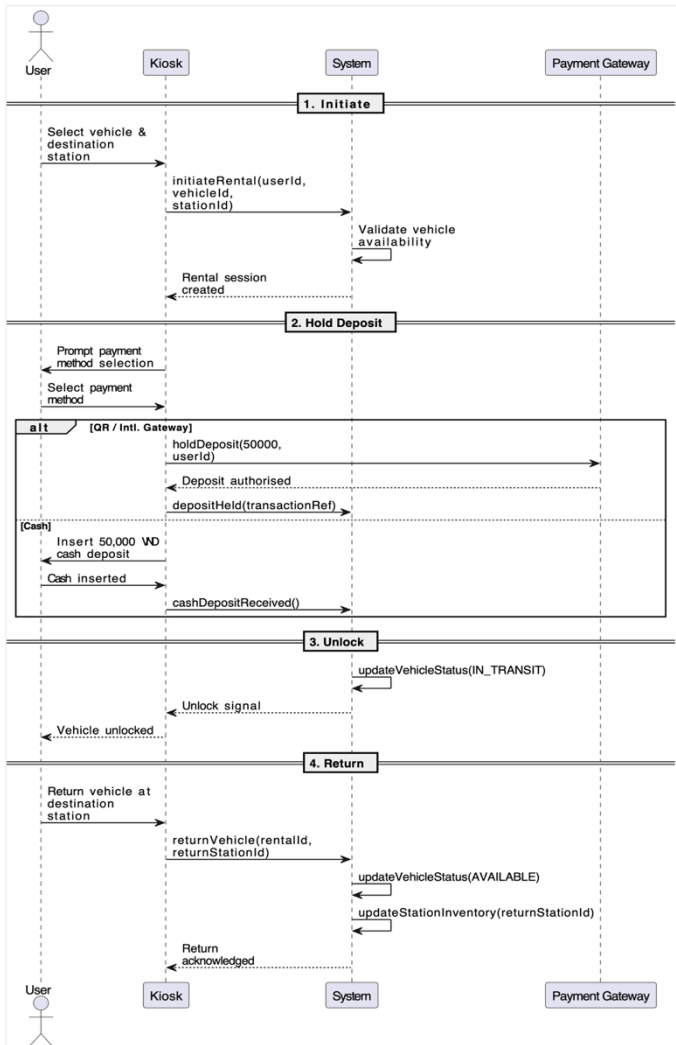
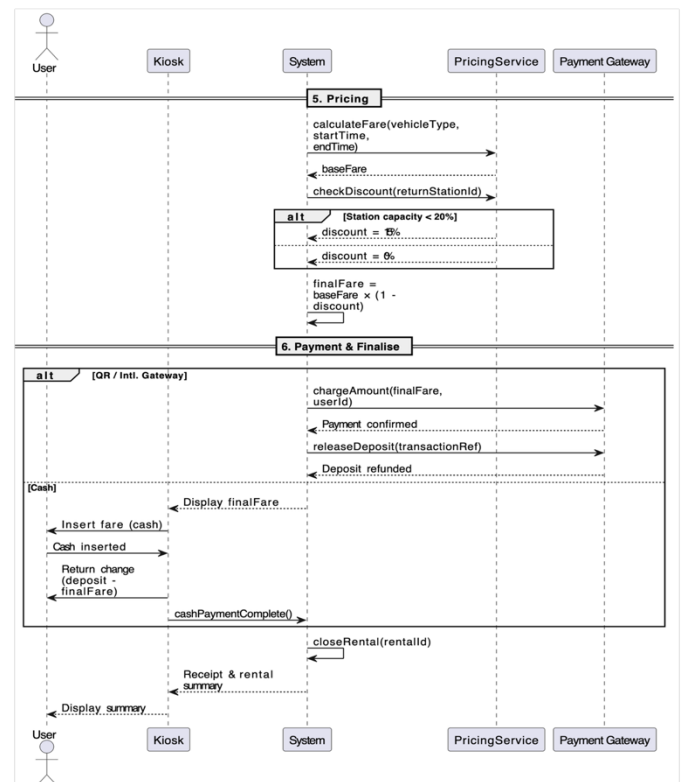
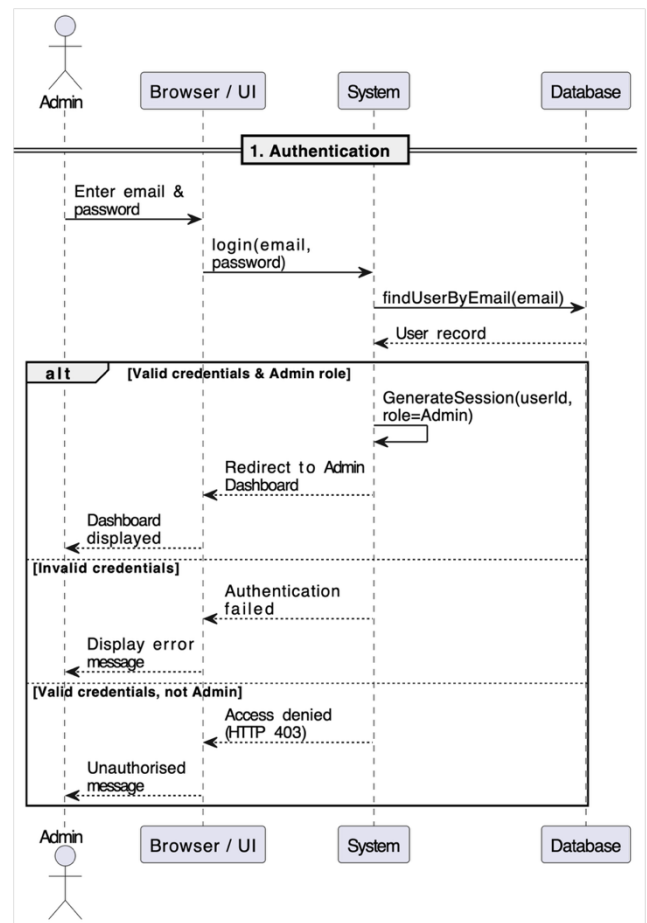


Figure 2.1.2: Sequence Diagram for Pricing Logic and Payment Finalization.
(Illustrates the collaboration between PricingService and Payment Gateway to handle discounts and finalize financial transactions.)



2.2. Admin Sequence Diagrams

Figure 2.2.1: Sequence Diagram for Admin Authentication. (Demonstrates the secure login workflow and role-based access verification for the management dashboard.)



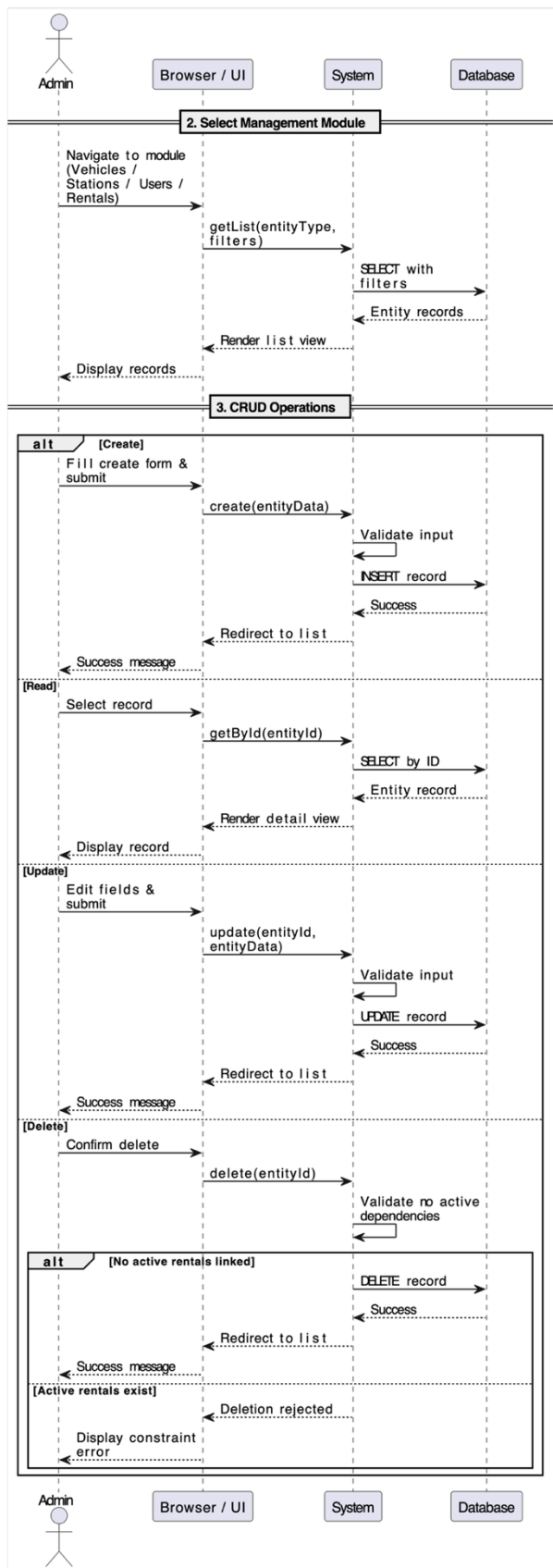
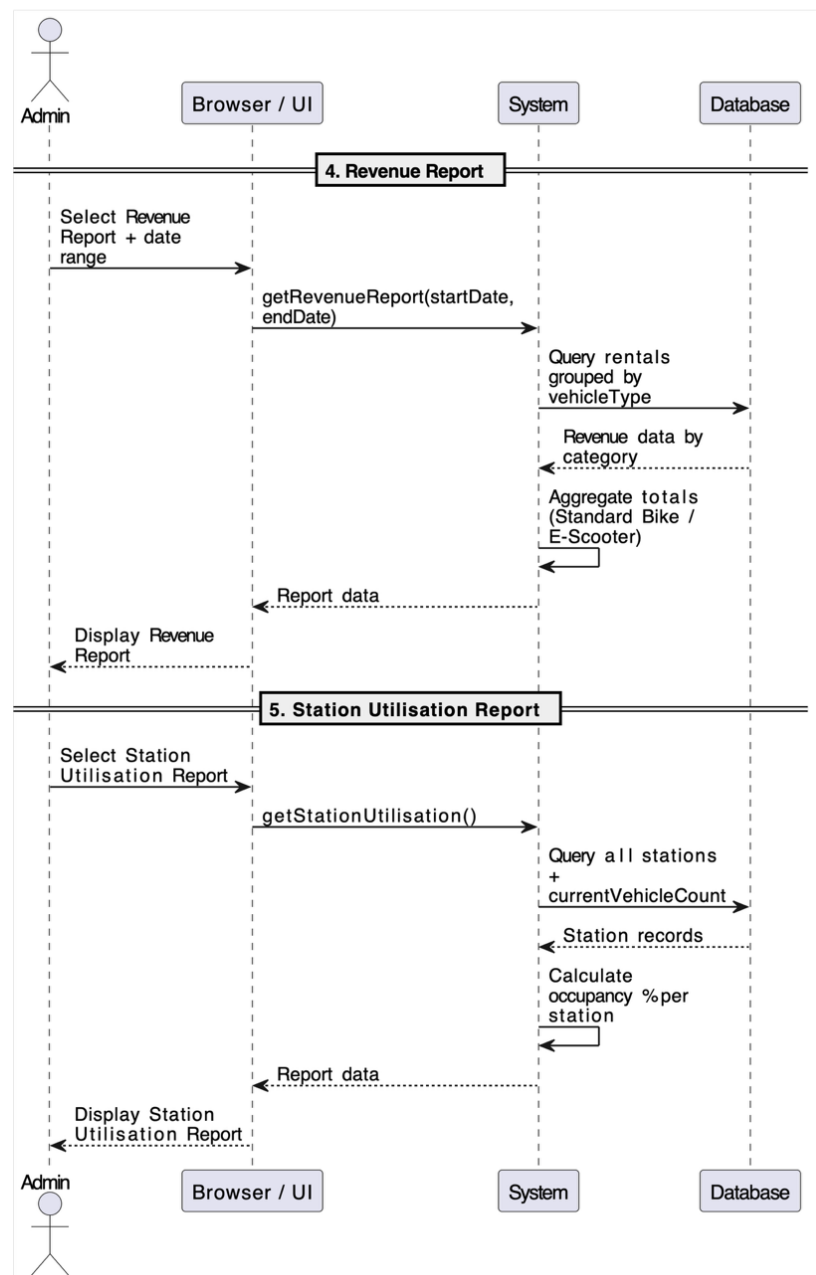


Figure 2.2.2: Sequence Diagram for Entity CRUD Operations. (Details the data flow for creating, retrieving, updating, and deleting records within the system's database.)

Figure 2.2.3: Sequence Diagram for System Reporting. (Shows the logic for querying and aggregating data to generate revenue and station utilization reports.)



4. Structural & CRC Design

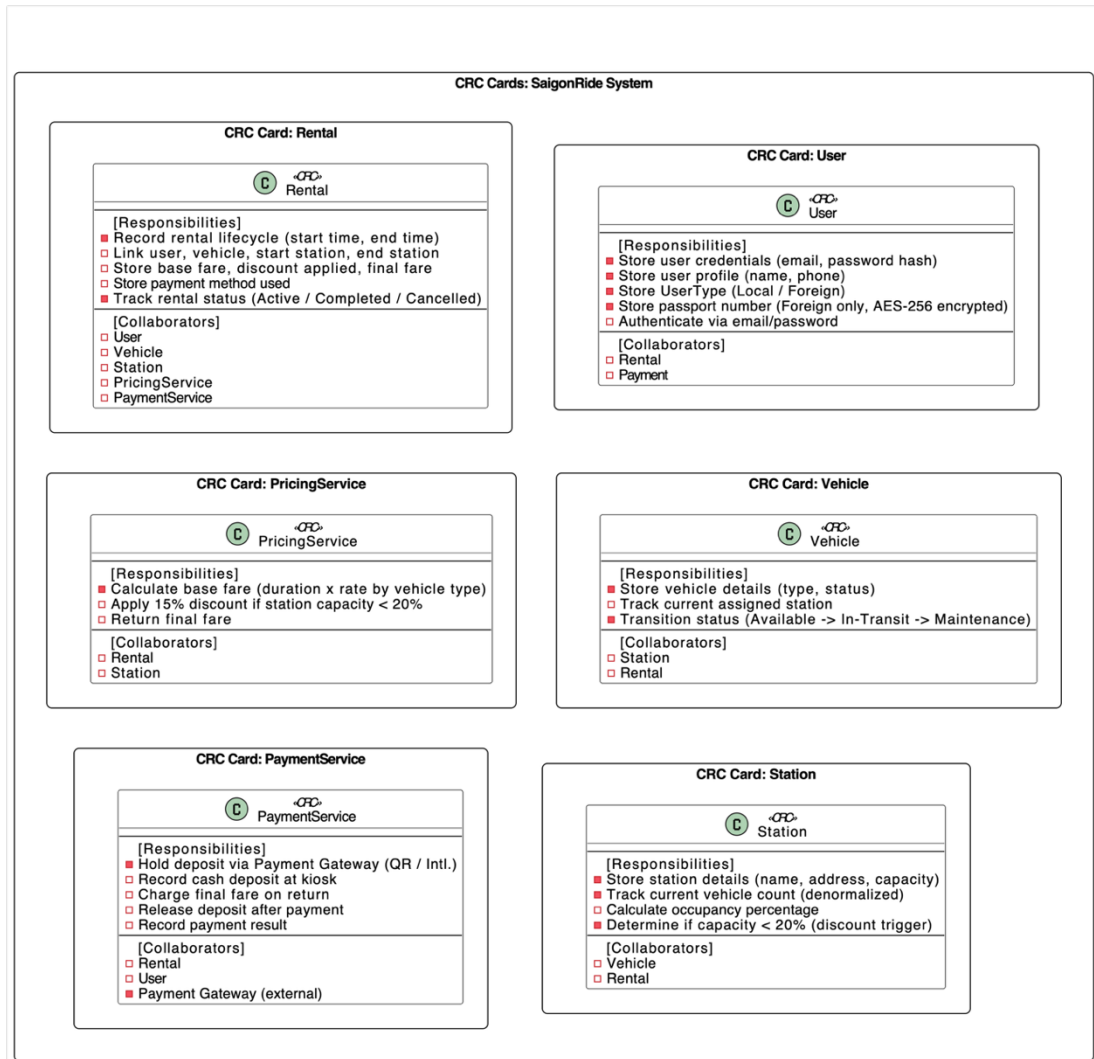


Figure 4.1: Class-Responsibility-Collaborator (CRC) Design Cards. (Defines the core responsibilities of key system classes and their collaborative relationships to fulfill business requirements.)

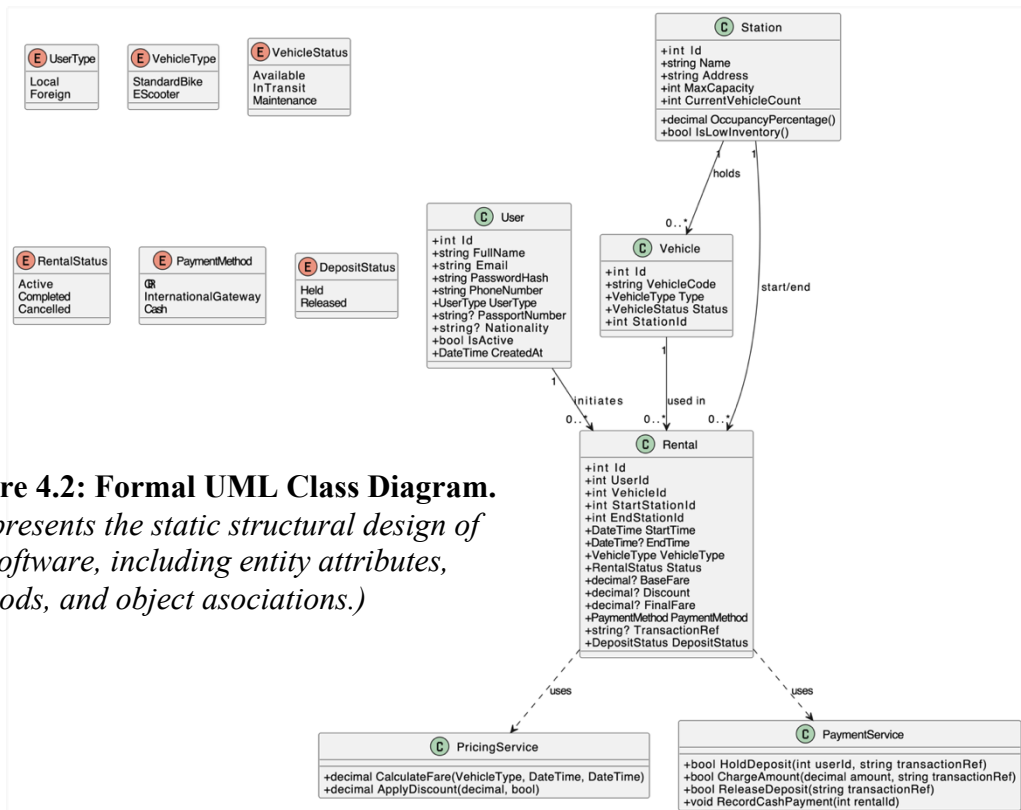
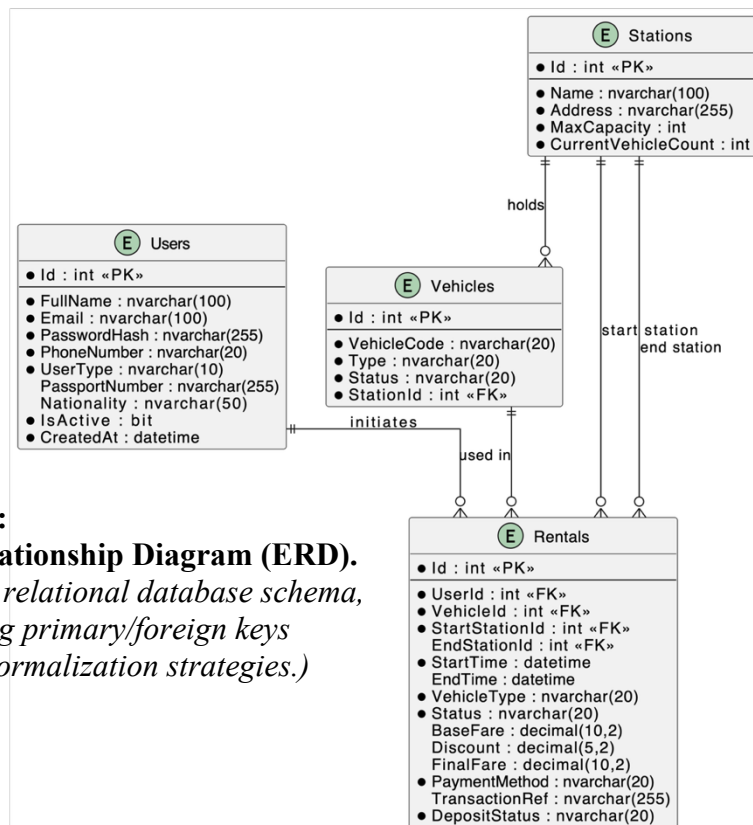


Figure 4.3:
Entity Relationship Diagram (ERD).
(Maps the relational database schema, highlighting primary/foreign keys and data normalization strategies.)



5. Architecture & Quality Design

5.1 Architectural Overview

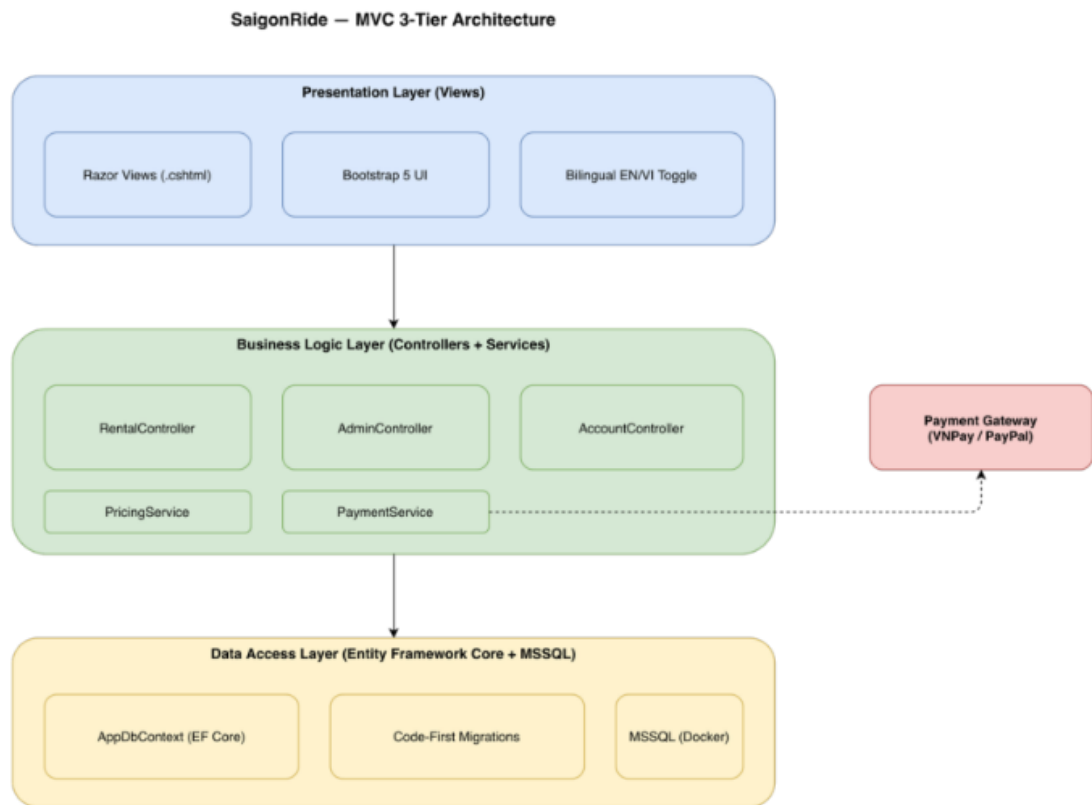


Figure 5.1: SaigonRide MVC 3-Tier Architectural Overview. *(Visualizes the separation of concerns across the Presentation, Business Logic, and Data Access layers.)*

SaigonRide is built on ASP.NET MVC Core following a strict 3-Tier architecture: Presentation (Razor Views + Bootstrap 5), Business Logic (Controllers + Services), and Data Access (Entity Framework Core Code-First + MSSQL). This separation ensures that no business logic leaks into Views and no direct database calls exist in Controllers, satisfying NFR-08.

5.2 SOLID Principles in Practice

Single Responsibility Principle (SRP) Each class owns exactly one concern. `PricingService` only calculates fares — it has no knowledge of payment processing or database access. `PaymentService` only handles deposit hold, charge, and release operations. Controllers only orchestrate request/response flow; they delegate all

computation to services. This directly satisfies NFR-10: `PricingService` is independently testable with zero UI dependencies.

Open/Closed Principle (OCP) The payment system is designed for extension without modification. New payment methods (e.g., MoMo, Apple Pay) can be added by implementing a new payment strategy without touching existing `PaymentService` logic. This is the Strategy Design Pattern applied to the payment gateway layer, as required by the case study specification.

Liskov Substitution Principle (LSP) `LocalCommuter` and `Foreign Tourist` are modelled as a single `User` entity with a `UserType` enum (Table-Per-Hierarchy). Both subtypes satisfy the same interface contract — authenticate, initiate rental, process payment — without breaking system behaviour. Either user type can be substituted anywhere a `User` is expected.

Interface Segregation Principle (ISP) Services expose only the methods their consumers need. `PricingService` exposes `CalculateFare()` and `ApplyDiscount()` — nothing else. Controllers are not forced to depend on methods they do not use, keeping the codebase low-coupling as required by NFR-08.

Dependency Inversion Principle (DIP) Controllers depend on service abstractions, not concrete implementations. `PricingService` and `PaymentService` are injected via ASP.NET Core's built-in DI container. This means controllers are never coupled to specific payment gateway implementations, satisfying NFR-09: VNPay timeout handling is isolated inside `PaymentService` without affecting `RentalController`.

5.3 How Architecture Satisfies NFRs

NFR	Principle	How
NFR-08 Maintainability	SRP + ISP	Zero business logic in Views; services are single-purpose
NFR-09 Reliability	DIP	Payment failures isolated in <code>PaymentService</code> , rolled back without affecting controller flow
NFR-10 Testability	SRP	<code>PricingService</code> is stateless and UI-independent — directly unit testable
NFR-01 Performance	SRP	<code>PricingService</code> performs pure computation, no DB calls, executes < 1.5s
NFR-05 Security	OCP + DIP	Auth middleware injected via DI; Admin routes protected by <code>[Authorize(Roles="Admin")]</code>
